

## EXPERIMENT-7

Aim : Jenkins architecture, installation, configuration, and pipelining primitives.

### 1. What is Jenkins?

**Jenkins** is an open-source automation server used to automate software development tasks such as **building, testing, and deploying applications**.

It is widely used in **CI/CD (Continuous Integration and Continuous Delivery/Deployment)**.

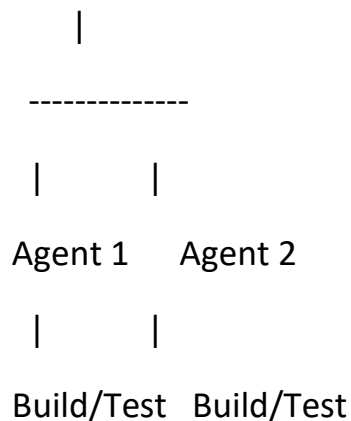
---

### 2. Jenkins Architecture

Jenkins mainly follows a **Controller–Agent architecture**.

- **Jenkins Controller:** Manages jobs, pipelines, configurations, and coordinates the work.
- **Jenkins Agent:** Executes the actual build, testing, and deployment tasks assigned by the controller.

Jenkins Controller



### 3. Jenkins Installation

Jenkins can be installed on a system and accessed through a **web browser**. After installation, the initial setup is performed by configuring Jenkins and installing the required plugins.

## 4. Jenkins Configuration

Jenkins configuration includes:

- Setting up the Jenkins environment.
  - Installing required plugins.
  - Managing users and permissions.
  - Configuring tools such as JDK and Git.
  - Creating and managing jobs/pipelines.
- 

## 5. Jenkins Pipeline

A **Jenkins Pipeline** is a sequence of automated steps used to build, test, and deploy an application.

Code



Build



Test



Deploy

Pipelines can be defined as code, making the automation repeatable and easier to manage.

---

## 6. Pipelining Primitives

Pipeline primitives are basic building blocks used to define Jenkins pipelines.

Important primitives include:

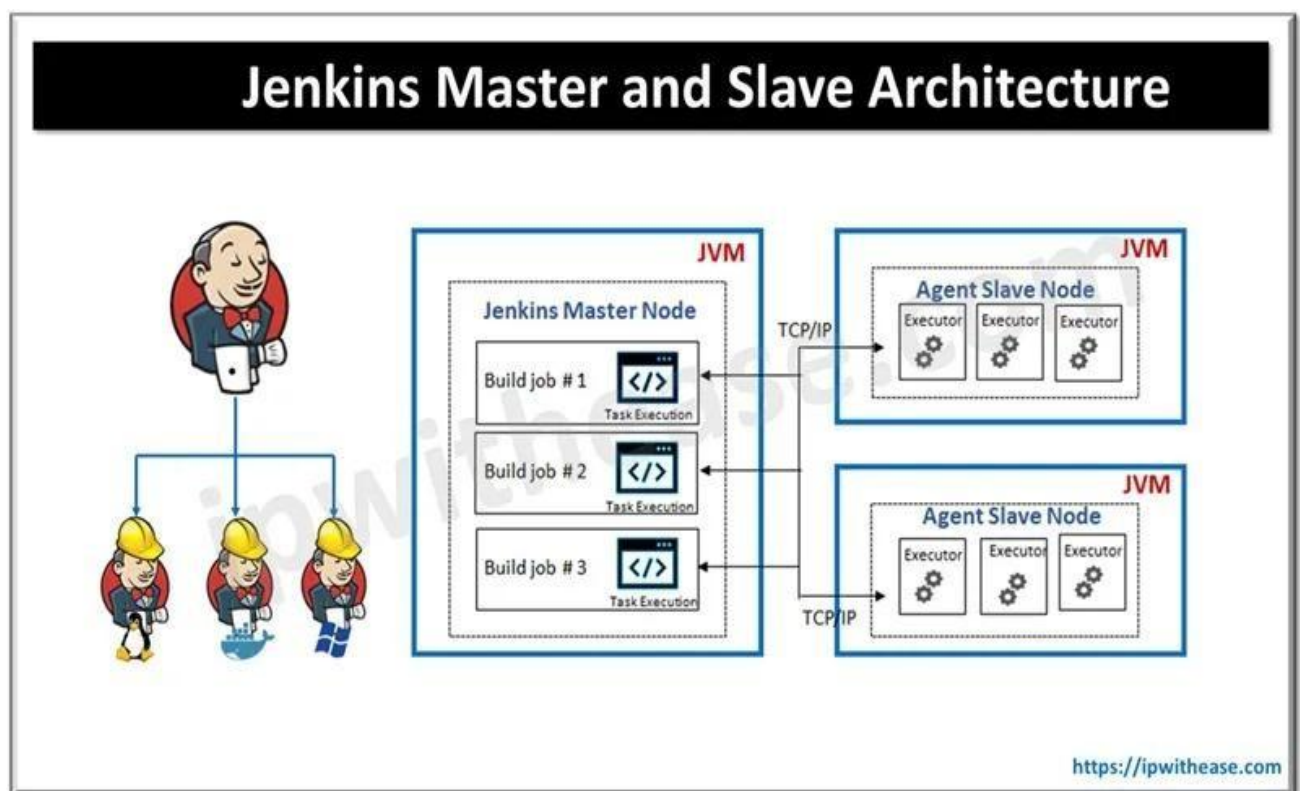
- **pipeline** – Defines the complete pipeline.

- **stage** – Represents a major phase such as Build, Test, or Deploy.
- **steps** – Contains the actual actions performed in a stage.
- **agent** – Specifies where the pipeline or stage should execute.

## Jenkins Architecture

Jenkins uses a distributed architecture to manage and execute build jobs efficiently across multiple machines.

- **Controller-Agent Architecture:** The Jenkins environment is built on a master-slave (now called controller-agent) model.
  - **Controller (Master):** This is the main server that manages the Jenkins user interface, configures jobs, and dispatches builds to agents for execution.
  - **Agents (Slaves):** These are worker machines that perform the actual build tasks assigned by the controller. Agents can be physical machines, virtual machines, or containers running on different operating systems like Windows, Linux, or Docker.



## Installation and Initial Configuration

### 1. System Requirements

Before installation, ensure your system has a compatible Java version (Java 8 or later is required) and sufficient hardware resources (at least 256MB RAM and 1GB disk space).

### 2. Download and Run Jenkins

Download the appropriate installer for your operating system (e.g., the .msi package for Windows) from the Jenkins website and run it. For a Docker-based installation, you would pull the official Jenkins image.

```
docker run -p 8080:8080 -p 50000:50000 jenkins/jenkins:lts
```

```
1 docker run -p 8080:8080 -p 50000:50000 jenkins/jenkins:lts
```

```
1 docker exec <container_id> cat /var/jenkins_home/secrets/initialAdminPassword
```

### 3. Unlock Jenkins

Getting Started

## Unlock Jenkins

To ensure Jenkins is securely set up by the administrator, a password has been written to the log ([not sure where to find it?](#)) and this file on the server:

```
C:\ProgramData\Jenkins\.jenkins\secrets\initialAdminPassword
```

Please copy the password from either location and paste it below.

Administrator password

Continue

## 4. Customize Jenkins

Getting Started

Customize Jenkins

Plugins extend Jenkins with additional features to support many different needs.

Install suggested plugins

Install plugins the Jenkins community finds most useful.

Select plugins to install

Select and install plugins most suitable for your needs.

Jenkins 2.452.1

## 5. Create First Admin user

Getting Started

Create First Admin User

Username

Password

Confirm password

Full name

E-mail address

Skip and continue as admin

Save and Continue

Jenkins 2.452.1

## 6. Instance Configuration

# Instance Configuration

Jenkins URL:

http://localhost:8080/

The Jenkins URL is used to provide the root URL for absolute links to various Jenkins resources. That means this value is required for proper operation of many Jenkins features including email notifications, PR status updates, and the BUILD\_URL environment variable provided to build steps.

The proposed default value shown is **not saved yet** and is generated from the current request, if possible. The best practice is to set this value to the URL that users are expected to use. This will avoid confusion when sharing or viewing links.

## 7. Ready to use

Dashboard >

+ New Item

📅 Build History

⚙️ Manage Jenkins

📌 My Views

Build Queue ▾

No builds in the queue.

Build Executor Status ▾

1 Idle

2 Idle

✎ Add description

Welcome to Jenkins!

This page is where your Jenkins jobs will be displayed. To get started, you can set up distributed builds or start building a software project.

Start building your software project

Create a job +

Set up a distributed build

Set up an agent 🖨

Configure a cloud ☁

Learn more about distributed builds ⓘ

## Jenkins Pipelining Primitives

### 1. Introduction to Pipelines

Jenkins Pipelines allow you to define your entire build, test, and deployment workflow as code using a suite of plugins. This provides flexibility and version control for your CI/CD process.

### 2. Declarative Pipeline Syntax

A declarative pipeline provides a structured and simplified syntax for defining your workflow. Its main primitives include:

**pipeline:** The main block that defines the entire workflow.

**agent:** Specifies where the pipeline will execute.

**stages:** A wrapper for all the different phases of the pipeline.

**stage:** Defines a specific phase, such as 'Build', 'Test', or 'Deploy'.

**steps:** Contains the actual commands or tasks to be run within a stage.

```
1 pipeline {
2   agent any
3   stages {
4     stage('Build') {
5       steps {
6         echo 'Building...'
7         // Add build steps here
8       }
9     }
10    stage('Test') {
11      steps {
```

```
        echo 'Testing...'
        // Add test steps here
      }
    }
    stage('Deploy') {
      steps {
        echo 'Deploying...'
        // Add deploy steps here
      }
    }
  }
}
```

## Example of Scripted Pipeline

```
1 node {
2   // Define a variable for the Git repository URL
3   def gitRepo = 'https://github.com/example/repo.git'
4   // Define a variable for the branch to build
5   def branch = 'main'
6   // Checkout code from the repository
7   stage('Checkout') {
8     echo 'Checking out code...'
9     git url: gitRepo, branch: branch
10  }
11  // Build the project using Maven
12  stage('Build') {
13    echo 'Building project...'
14    sh 'mvn clean install'
15  }
16 }
```